

## Code Explanation

# Code Explanation: AWS Glue PySpark ETL Job Unit Tests  
This test suite validates an AWS Glue ETL job that processes customer and order data. Below is a step-by-step breakdown:---

### **\*\*1. Test Setup and Fixtures\*\***

#### **\*\*Spark Session Fixture\*\***

```
@pytest.fixture(scope="session")def spark():
```

- Creates a local Spark session for testing with 2 cores- Sets log level to ERROR to reduce noise-
- Session-scoped: created once and reused across all tests- Cleans up after all tests complete

#### **\*\*Sample Data Fixtures\*\***

```
@pytest.fixturedef sample_customer_data():
```

- Provides clean customer records (5 customers with ID, name, email, region)- Provides clean order records (5 orders with ID, item, price, quantity, date, customer ID)- Provides dirty data with nulls and duplicates for testing data quality functions---

### **\*\*2. Initialization Tests\*\***

#### **\*\*Test Spark Context Initialization\*\***

```
def test_initialize_spark_contexts():
```

- Verifies that `initialize_spark_contexts()` returns valid Spark and Glue contexts- Ensures the `SparkSession` object is properly instantiated

#### **\*\*Test Job Parameters Loading\*\***

```
def test_get_job_parameters():
```

- Validates that YAML configuration loads correctly- Checks for required sections: inputs, outputs, catalog, hudi, processing- Verifies specific paths:
  - Customer input: ``s3://adif-sdlc/sdlc_wizard/customerdata/``
  - Order input: ``s3://adif-sdlc/sdlc_wizard/orderdata/``
  - Customer output: ``s3://adif-sdlc/curated/sdlc_wizard/customer/``
- Confirms format specifications (CSV for input, Hudi for output)---

### **\*\*3. Validation Tests\*\***

#### **\*\*Test S3 Access Validation\*\***

```
def test_validate_s3_access(spark):
```

- Confirms `validate_s3_access()` returns ``True`` for valid S3 paths- Returns ``False`` for local file paths or non-S3 URIs- Does not actually connect to S3 (validation is path-based)

#### **\*\*Test Schema Definitions\*\***

```
def test_get_customer_schema():def test_get_order_schema():
```

- Validates customer schema has 4 fields: custid, name, emailid, region- Validates order schema has 6 fields: orderid, itemname, priceperunit, qty, date, custid- Ensures schemas are proper PySpark StructType objects---

## **\*\*4. Data Quality Tests\*\***

### **\*\*Test Null Removal\*\***

```
def test_clean_dataframe_removes_nulls(spark, sample_customer_data_with_nulls):
```

- Creates DataFrame with actual NULL values and "Null" strings- Calls `clean\_dataframe()` with null\_values list `["Null", "NULL", "null"]`- Verifies that rows containing nulls or null-like strings are removed- Expected result: only 1 valid unique record remains

### **\*\*Test Duplicate Removal\*\***

```
def test_clean_dataframe_removes_duplicates(spark, sample_customer_data):
```

- Adds a duplicate record to clean data- Verifies `clean\_dataframe()` removes duplicates- Expected result: 5 unique records (duplicate removed)

### **\*\*Test Case-Sensitive Null Matching\*\***

```
def test_clean_dataframe_case_sensitive_null_matching(spark):
```

- Tests with "Null", "NULL", and "null" strings- Configures to match only "Null" (case-sensitive)- Verifies only exact matches are removed- Expected: "NULL" and "null" remain, "Null" removed---

## **\*\*5. SCD Type 2 (Slowly Changing Dimension) Tests\*\***

### **\*\*Test SCD2 Metadata Columns\*\***

```
def test_apply_scd2_creates_metadata_columns(spark, ...):
```

- Verifies `apply\_scd2()` adds required tracking columns: - `isactive`: Boolean flag for current records - `startdate`: When record became active - `enddate`: When record expired (NULL for active) - `opts`: Additional options/metadata- Confirms all initial records are active with NULL enddate

### **\*\*Test SCD2 Join Logic\*\***

```
def test_apply_scd2_joins_customer_and_order(spark, ...):
```

- Validates inner join between customer and order DataFrames on custid- Expected: 5 joined records (matching the 5 orders)- Verifies both customer fields (name, emailid, region) and order fields (orderid, itemname, priceperunit) are present

### **\*\*Test SCD2 Composite Business Key\*\***

```
def test_scd2_composite_business_key(spark):
```

- Tests scenario where one customer (C001) has multiple orders (O001, O002)- Verifies composite key (CustId + OrderId) creates separate records- Expected: 2 records for C001, one per order---

## **\*\*6. Aggregation Tests\*\***

### **\*\*Test Customer Spend Calculation\*\***

```
def test_aggregate_customer_spend_calculates_totals(spark, ...):
```

- Validates `aggregate_customer_spend()` creates summary metrics: - `total_spend`: Sum of (priceperunit × qty) - `order_count`: Number of orders - `total_quantity`: Sum of quantities - `max_price`: Highest price - `min_price`: Lowest price- Verifies calculation for O001:  $999.99 \times 1 = 999.99$

### **\*\*Test Aggregation Grouping\*\***

```
def test_aggregate_customer_spend_groups_by_date(spark, ...):
```

- Confirms aggregation groups by (custid, date)- Expected: 5 unique customer-date combinations- Ensures proper grouping columns are present---

## **\*\*7. Data I/O Tests (Mocked)\*\***

### **\*\*Test DataReader for CSV\*\***

```
def test_data_reader_csv(spark, sample_customer_data):
```

- Mocks `_read_csv()` method to avoid actual S3 reads- Verifies `read_data()` returns DataFrame with correct row count- Tests CSV format reading with schema

### **\*\*Test DataReader for Parquet\*\***

```
def test_data_reader_parquet(spark, sample_customer_data):
```

- Mocks `_read_parquet()` method- Validates Parquet reading without schema (inferred)

### **\*\*Test DataWriter for Parquet/CSV\*\***

```
def test_data_writer_parquet(spark, sample_customer_data):def test_data_writer_csv(spark, sample_customer_data):
```

- Mocks write methods to avoid actual S3 writes- Verifies `write_data()` calls the appropriate internal method- Uses `assert_called_once()` to confirm execution

### **\*\*Test DataWriter for Hudi\*\***

```
def test_data_writer_hudi(spark, sample_customer_data):
```

- Mocks `_write_hudi()` method- Provides required Hudi options: - `hoodie.table.name`: Table identifier - `hoodie.datasource.write.recordkey.field`: Primary key (custid) - `hoodie.datasource.write.precombine.field`: Version field (name)- Verifies method is called with options

### **\*\*Test Hudi Options Requirement\*\***

```
def test_data_writer_hudi_requires_options(spark, sample_customer_data):
```

- Validates that calling `write_data()` with format="hudi" but no options raises `ValueError`- Error message: "Hudi options required"---

## **\*\*8. Additional Validation Tests\*\***

### **\*\*Test YAML Path Handling\*\***

```
def test_yaml_paths_are_opaque_strings():
```

- Confirms configuration paths are treated as strings- Verifies paths start with `s3://` prefix- Does not attempt actual path validation or dereferencing

### **\*\*Test Column Normalization\*\***

```
def test_column_normalization(spark, sample_customer_data):
```

- Creates DataFrame with mixed-case column names (CustId, Name, EmailId, Region)- Normalizes to lowercase using `toDF(\*[c.lower() for c in df.columns])`- Verifies all columns are lowercase (custid, name, emailid, region)---

### **\*\*Key Testing Patterns Used\*\***

1. **\*\*Mocking\*\***: Uses `unittest.mock.patch` to avoid actual S3 I/O operations2. **\*\*Fixtures\*\***: Reusable test data and Spark session3. **\*\*Assertions\*\***: Validates row counts, column presence, data values, and method calls4. **\*\*Error Testing\*\***: Uses `pytest.raises` to verify exception handling5. **\*\*Isolation\*\***: Each test is independent and uses fresh data---

### **\*\*Test Execution\*\***

Run with:

```
pytest test_job.py -v
```

The `-v` flag provides verbose output showing each test result.